

The University of Faisalabad

Computer vision

Lab manual

Registration NUMBER:

2023-BSAI-037 (6A)

Submitted to:

Ms. Esha Nawaz

Submitted by:

QANDEEL ASIM

Department: Artificial intelligence

Table of content

LAB 1:Introduction to Computer Vision	3
Lab 2: Introduction to Deep Learning for Computer Vision	6
Lab 3 : Corner Detection and Fourier Transform	9
Lab 4 fourier transform:	15
Lab 5: Corner and Edge Detection via Eigenvalue Analysis	21
Lab 6: Image Thresholding	25
Lab 7: Unsupervised Image Segmentation	28
Lab 8: Felzenszwalb Graph-Based Segmentation	34
Lab 9: Image Segmentation Pipeline	36
Lab 10: CIFAR-10 Image Classification using CNN	40
Lab 11: Image Classification using Pre-trained CNN (MobileNetV2)	42
Lab 12: Simple Linear Regression with Scikit-Learn	48
Lab 13: Feature Extraction Using SIFT in OpenCV	50
Lab 14: HOG Feature Extraction for Image Analysis	52

LAB 1: Introduction to Computer Vision

Lab 1 introduces the basic concepts of Computer Vision and explains that images are simply grids of numerical values (NumPy arrays). Students learn how to load, display, and examine images using OpenCV and Matplotlib, along with basic operations such as converting images to grayscale and performing pixel-level modifications. The lab also explores two key challenges in computer vision: **illumination changes** and **occlusion**. Students observe how variations in lighting affect pixel values and how hidden or missing parts of an image make analysis more difficult. Next, students are introduced to a classical technique called **Canny Edge Detection** for detecting object boundaries. By applying it to images with different brightness levels, they learn that fixed thresholds do not perform well under varying conditions. The lab concludes with a mini-task where students apply all techniques to their own image and reflect on the results. Overall, the focus is on hands-on learning to build a clear understanding of the challenges in computer vision.

Viva related question:

- **Why do edges disappear in dark images?** Edge detection (like Canny) works by calculating the intensity gradient (the difference in pixel values) between neighboring pixels. In dark images, the range of numerical values is compressed (e.g., all pixels are between 0 and 20). Because the "contrast" or mathematical difference between an object and its background becomes very small, the algorithm cannot find a threshold high enough to define an edge.
- **Why is occlusion difficult?** A computer "sees" an image as a static grid of numbers. If a black box covers an object, the original numbers are replaced by zeros. Unlike humans, who use "amodal completion" (our brain fills in the gaps based on experience), a classical machine vision algorithm sees the object as physically broken or as two completely different shapes.
- **What information is lost in grayscale?** Grayscale collapses three dimensions of data (Red, Green, and Blue) into a single intensity value. You lose **chromaticity**. For example, a red apple and a green leaf might have the exact same "brightness" or luminance value; in grayscale, they would look identical, making it impossible to distinguish them based on color alone.

LOAD LIBRARY:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

1. Load Your Image

```
: image_path = 'download (2).jfif'
img_bgr = cv2.imread(image_path)

if img_bgr is None:
    print("Error: Image not found. Check the file path.")
else:
    # Convert BGR to RGB for correct Matplotlib display
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

2. Convert to Grayscale

```
: gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)
```

3. Applying Brightness Change, Occlusion & Edge Detection (Canny)

```
: bright_img = cv2.convertScaleAbs(img_rgb, alpha=1.2, beta=30)
occluded_img = img_rgb.copy()
cv2.rectangle(occluded_img, (100, 100), (300, 300), (0, 0, 0), -1)
edges = cv2.Canny(gray, 100, 200)
```

4. Visualize All Results

```
plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("1. Original RGB")
plt.axis('off')

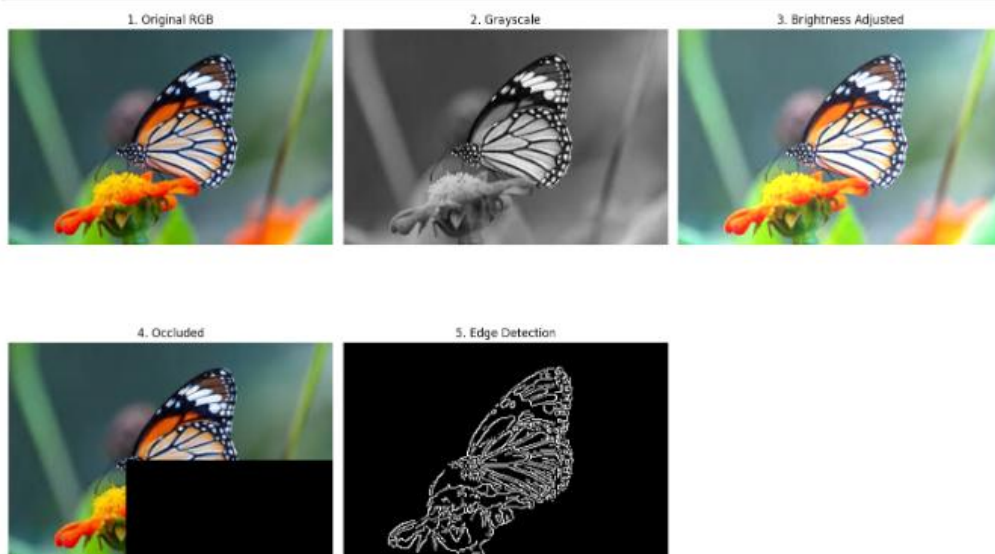
plt.subplot(2, 3, 2)
plt.imshow(gray, cmap='gray')
plt.title("2. Grayscale")
plt.axis('off')

plt.subplot(2, 3, 3)
plt.imshow(bright_img)
plt.title("3. Brightness Adjusted")
plt.axis('off')

plt.subplot(2, 3, 4)
plt.imshow(occluded_img)
plt.title("4. Occluded")
plt.axis('off')

plt.subplot(2, 3, 5)
plt.imshow(edges, cmap='gray')
plt.title("5. Edge Detection")
plt.axis('off')

plt.tight_layout()
plt.show()
```



Lab 2: Introduction to Deep Learning for Computer Vision

Lab 2 introduces deep learning and shows how it improves upon classical computer vision methods. Building on Lab 1, where students learned that images are numerical arrays and that lighting and occlusion create challenges, this lab demonstrates how Convolutional Neural Networks (CNNs) can automatically learn features from data.

Students use PyTorch to load and preprocess images, convert them to grayscale, and organize them using ImageFolder and DataLoader. A simple CNN model is created with one convolutional layer and one fully connected layer to classify images into two categories.

The model is trained for a few epochs using cross-entropy loss and the Adam optimizer, and its performance is evaluated using accuracy on a test set.

This lab highlights that, unlike traditional methods, CNNs do not require manual feature extraction. Instead, they learn important patterns directly from data, helping to handle issues like lighting changes and occlusion.

Load library:

```
: import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
```

1. Setup & Preprocessing

```
: transform = transforms.Compose([
    transforms.Grayscale(),
    transforms.Resize((64, 64)),
    transforms.ToTensor()
])

train_path = r"E:\Folio 1040 Hp\Desktop\CV\simple_cv_dataset\train"
test_path = r"E:\Folio 1040 Hp\Desktop\CV\simple_cv_dataset\test"

train_data = datasets.ImageFolder(train_path, transform=transform)
test_data = datasets.ImageFolder(test_path, transform=transform)
```

2. Define the Simple Neural Network (MLP)

```
class SimpleMLP(nn.Module):
    def __init__(self):
        super().__init__()
        # Input size: 1 channel * 64 * 64 pixels = 4096
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(4096, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 2) # 2 Classes: Circle and Square

    def forward(self, x):
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

3. Define the CNN

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 8, kernel_size=3)
        self.relu = nn.ReLU()
        self.fc = nn.Linear(8*62*62, 2)

    def forward(self, x):
        x = self.relu(self.conv(x))
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x
```

4. Training Function

```
def train_model(model, train_loader, epochs=10):
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)
    model.train()

    for epoch in range(epochs):
        running_loss = 0.0
        for images, labels in train_loader:
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()
    return model
```

5. Evaluation Function

```
] : def evaluate_model(model, test_loader):
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in test_loader:
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    return 100 * correct / total

train_loader = DataLoader(train_data, batch_size=8, shuffle=True)
test_loader = DataLoader(test_data, batch_size=8)

cnn_model = SimpleCNN()
mlp_model = SimpleMLP()

print("Training CNN...")
train_model(cnn_model, train_loader)
cnn_acc = evaluate_model(cnn_model, test_loader)

print("Training Simple MLP...")
train_model(mlp_model, train_loader)
mlp_acc = evaluate_model(mlp_model, test_loader)

Training CNN...
Training Simple MLP...
```

6. Final Results

```
print("\n--- Final Performance Comparison ---")
print(f"CNN Accuracy: {cnn_acc:.2f}%")
print(f"MLP Accuracy: {mlp_acc:.2f}%")
```

```
--- Final Performance Comparison ---
CNN Accuracy: 100.00%
MLP Accuracy: 100.00%
```


Lab 3 : Corner Detection and Fourier Transform

This combined lab introduces two important classical computer vision techniques: **Harris Corner Detection** and the **Fourier Transform**. These methods represent traditional approaches used before deep learning, focusing on understanding image structure in both spatial and frequency domains. In Lab 3, students apply the Harris Corner Detector to identify key points in an image where intensity changes sharply. The image is converted to grayscale and processed using OpenCV, and detected corners are highlighted. This method is useful in tasks like image matching, tracking, and 3D reconstruction. Lab 4 focuses on the Fourier Transform, where images are converted into the frequency domain using FFT. Students visualize the frequency spectrum and apply filters to understand image components. A low-pass filter is used to blur the image by keeping smooth regions, while a high-pass filter highlights edges and fine details. Overall, these labs demonstrate how traditional techniques analyze images using mathematical methods, forming the foundation for many modern computer vision applications.

TASK 1: Harris Corner Detection

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

1. Load and prepare the image

```
: image_path = 'BUIDINGG.jfif'
img = cv2.imread(image_path)

if img is None:
    print("Error: Image not found!")
else:
    # Keep a copy for processing
    original_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Harris requires Grayscale in Float32 format
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = np.float32(gray)
```

2. Apply Harris Corner Detection

```
# Parameters: (image, blockSize, ksize, k)
# k is the free parameter (usually 0.04 to 0.06)
dst = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)
```

3. Dilate to make corners visible, Thresholding and Marking, Low vs High Sensitivity

```
dst = cv2.dilate(dst, None)

marked_img = original_rgb.copy()
marked_img[dst > 0.01 * dst.max()] = [255, 0, 0] # Mark corners Red

# Mark only the strongest corners (10% of max)
strong_corners = original_rgb.copy()
strong_corners[dst > 0.1 * dst.max()] = [0, 255, 0] # Mark strong corners Green
```

4. Visualization

```
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(original_rgb)
plt.title("Original Image")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(marked_img)
plt.title("Corners (Threshold 1%)")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(strong_corners)
plt.title("Strongest Corners (Threshold 10%)")
plt.axis('off')

plt.tight_layout()
plt.show()
```

Original Image



Corners (Threshold 1%)



Strongest Corners (Threshold 10%)



TASK 2: Image Processing in Transform

Domain (Fourier Transform)

1. Load Image in Grayscale

```
img = cv2.imread('BUIDINGG.jfif', 0)
```

2. Perform Fourier Transform, Calculate Magnitude Spectrum for visualization

```
# fft2 converts image to frequency domain
f = np.fft.fft2(img)
# fftshift moves the zero-frequency (Low Frequency) to the center
fshift = np.fft.fftshift(f)

magnitude_spectrum = 20 * np.log(np.abs(fshift))
```

3. Prepare Masks for Filtering

```
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2 # Center coordinates
radius = 30 # Size of the filter square

lpf_mask = np.zeros((rows, cols), np.uint8) #Low Pass Filter (LPF) Mask
lpf_mask[crow-radius : crowd+radius, ccol-radius : ccol+radius] = 1
lpf_fshift = fshift * lpf_mask

hpf_mask = np.ones((rows, cols), np.uint8) #High Pass Filter (HPF) Mask
hpf_mask[crow-radius : crowd+radius, ccol-radius : ccol+radius] = 0
hpf_fshift = fshift * hpf_mask
```

4. Inverse Fourier Transform

```
def inverse_fft(shifted_data):  
    f_ishift = np.fft.ifftshift(shifted_data)  
    img_back = np.fft.ifft2(f_ishift)  
    return np.abs(img_back)  
  
img_lpf = inverse_fft(lpf_fshift)  
img_hpf = inverse_fft(hpf_fshift)
```

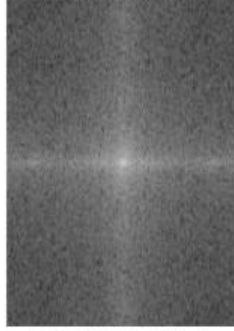
5. Final Visualization

```
plt.figure(figsize=(16, 8))  
  
plt.subplot(2, 3, 1)  
plt.imshow(img, cmap='gray')  
plt.title("1. Original Image")  
plt.axis('off')  
  
plt.subplot(2, 3, 2)  
plt.imshow(magnitude_spectrum, cmap='gray')  
plt.title("2. Frequency Spectrum")  
plt.axis('off')  
  
plt.subplot(2, 3, 3)  
plt.imshow(lpf_mask, cmap='gray')  
plt.title("3. LPF Mask (Center Pass)")  
plt.axis('off')  
  
plt.subplot(2, 3, 4)  
plt.imshow(img_lpf, cmap='gray')  
plt.title("4. Result: Low Pass (Blurred)")  
plt.axis('off')  
  
plt.subplot(2, 3, 5)  
plt.imshow(hpf_mask, cmap='gray')  
plt.title("5. HPF Mask (Center Block)")  
plt.axis('off')  
  
plt.subplot(2, 3, 6)  
plt.imshow(img_hpf, cmap='gray')  
plt.title("6. Result: High Pass (Edges)")  
plt.axis('off')  
  
plt.tight_layout()  
plt.show()
```

1. Original Image



2. Frequency Spectrum



3. LPF Mask (Center Pass)



4. Result: Low Pass (Blurred)



5. HPF Mask (Center Block)



6. Result: High Pass (Edges)



Lab 4 fourier transform:

Lab 4 focuses on the Fourier Transform, where images are converted into the frequency domain using FFT. Students visualize the frequency spectrum and apply filters to understand image components. A low-pass filter is used to blur the image by keeping smooth regions, while a high-pass filter highlights edges and fine details. Overall, these labs demonstrate how traditional techniques analyze images using mathematical methods, forming the foundation for many modern computer vision applications.

Interactive Harris Classification (Harris Corner Detection)

Step 1: Upload Image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image_path = "BUILDING.jfif"

img = cv2.imread(image_path)

if img is None:
    print("Error: Image not found. Check the file path.")
else:
    # Convert to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = np.float32(gray)

    plt.figure(figsize=(6,6))
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title("Uploaded Image")
    plt.axis("off")
    plt.show()
```

Uploaded Image



Step 2: Compute Gradients and Structure Matrix Components

```
Ix = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
Iy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
Ixx = Ix * Ix
Iyy = Iy * Iy
Ixy = Ix * Iy
Sxx = cv2.GaussianBlur(Ixx, (3,3), 1)
Syy = cv2.GaussianBlur(Iyy, (3,3), 1)
Sxy = cv2.GaussianBlur(Ixy, (3,3), 1)
```

Step 3: Compute Eigenvalues and Classify

```
height, width = gray.shape

corner_img = img.copy()
edge_img = img.copy()
flat_img = img.copy()

threshold = 0.01 * Sxx.max()

for y in range(height):
    for x in range(width):

        M = np.array([[Sxx[y,x], Sxy[y,x]],
                      [Sxy[y,x], Syy[y,x]]])

        eigenvalues, _ = np.linalg.eig(M)
        lambda1, lambda2 = eigenvalues

        if lambda1 > threshold and lambda2 > threshold:
            corner_img[y,x] = [0,0,255] # Red (corner)

        elif (lambda1 > threshold and lambda2 <= threshold) or \
             (lambda2 > threshold and lambda1 <= threshold):
            edge_img[y,x] = [255,0,0] # Blue (edge)

        else:
            flat_img[y,x] = [0,255,0] # Green (flat)
```


Step 5: Show Results

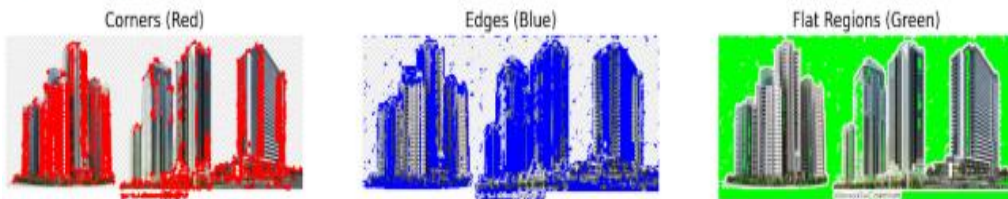
```
plt.figure(figsize=(15,5))

# Corners
plt.subplot(1,3,1)
plt.title("Corners (Red)")
plt.imshow(cv2.cvtColor(corner_img, cv2.COLOR_BGR2RGB))
plt.axis("off")

# Edges
plt.subplot(1,3,2)
plt.title("Edges (Blue)")
plt.imshow(cv2.cvtColor(edge_img, cv2.COLOR_BGR2RGB))
plt.axis("off")

# Flat regions
plt.subplot(1,3,3)
plt.title("Flat Regions (Green)")
plt.imshow(cv2.cvtColor(flat_img, cv2.COLOR_BGR2RGB))
plt.axis("off")

plt.show()
```



Lab Task

```
def harris_detection(image_path, threshold=0.02, blur=False):
```

```
    img = cv2.imread(image_path)
```

```
    if img is None:
```

```
        print("Image not found:", image_path)
```

```
        return
```

```
    if blur:
```

```
        img = cv2.GaussianBlur(img, (7,7), 2)
```

```
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
    gray = np.float32(gray)
```

```
    # Gradients
```

```
    Ix = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```

```
    Iy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
```

```
    # Structure matrix
```

```
    Ixx = Ix * Ix
```

```
    Iyy = Iy * Iy
```

```
    Ixy = Ix * Iy
```

```
    Sxx = cv2.GaussianBlur(Ixx, (3,3), 1)
```

```
    Syy = cv2.GaussianBlur(Iyy, (3,3), 1)
```

```
    Sxy = cv2.GaussianBlur(Ixy, (3,3), 1)
```

```
    # Harris response
```

```
    k = 0.04
```

```
    R = (Sxx * Syy - Sxy**2) - k * (Sxx + Syy)**2
```

```
    # Thresholding
```

```
    corner_img = img.copy()
```

```
    corner_mask = R > threshold * R.max()
```

```
    corner_img[corner_mask] = [0,0,255] # Red corners
```

```
    corner_count = np.sum(corner_mask)
```

```
    # Show result
```

```
    plt.figure(figsize=(6,6))
```

```
    plt.imshow(cv2.cvtColor(corner_img, cv2.COLOR_BGR2RGB))
```

```
    plt.title(f"Corners Detected: {corner_count}")
```

```
    plt.axis("off")
```

```
    plt.show()
```

```
    print("Total Corners:", corner_count)
```

```
    return corner_count
```

Building Image

```
: print("Building Image")
building_corners = harris_detection("BUILDING.jfif", threshold=0.02)
```

Building Image

Corners Detected: 5884



Total Corners: 5884

Road Image

```
] print("Road Image")
road_corners = harris_detection("ROAD IMG.jfif", threshold=0.02)
```

Road Image

Corners Detected: 1036



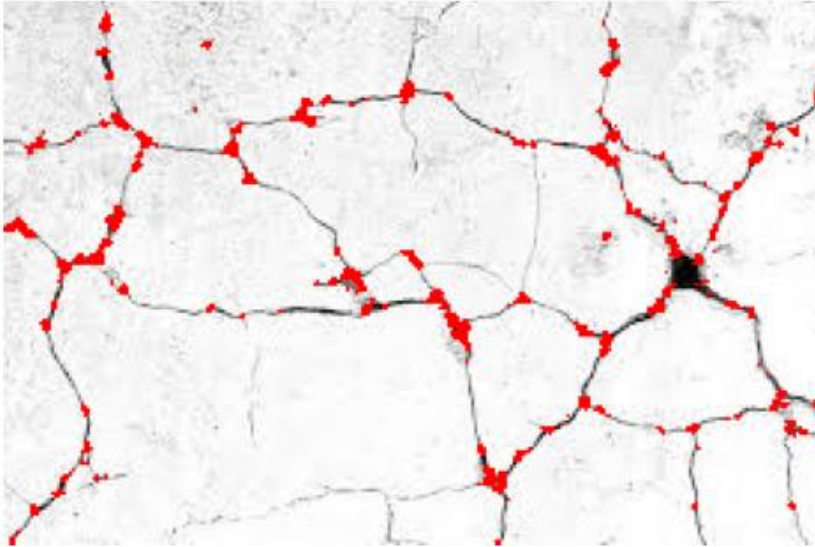
Total Corners: 1036

Crack Image

```
: print("Crack Image")  
crack_corners = harris_detection("CRACK IMG.jfif", threshold=0.02)
```

Crack Image

Corners Detected: 1440



Total Corners: 1440

Lab 5: Corner and Edge Detection via Eigenvalue Analysis

Lab 5 focuses on implementing the Harris Corner Detection algorithm from scratch, helping students understand how it works mathematically rather than relying on built-in functions. The image is first converted to grayscale, and Sobel operators are used to compute gradients in both directions. These gradients are then used to form the structure tensor, which is smoothed using a Gaussian filter. For each pixel, eigenvalues of the structure matrix are calculated to classify regions. If both eigenvalues are large, the point is a corner; if one is large and the other small, it is an edge; and if both are small, it represents a flat region. The results are displayed using color coding: corners in red, edges in blue, and flat regions in green. This lab helps students understand that corner detection is based on mathematical concepts like gradients and eigenvalues, and allows them to experiment with thresholds to see how detection changes. It strengthens their understanding of earlier concepts and prepares them for advanced computer vision tasks.

Real- Time Object Detection with OpenCV and MobileNet SSD

```
import cv2
import numpy as np
import os
import urllib.request
import matplotlib.pyplot as plt
```

2. Load Model

```
import urllib.request
import os

# Create the folder if it doesn't exist
os.makedirs("models", exist_ok=True)

# Correct URLs for the MobileNet SSD files
urls = {
    "MobileNetSSD_deploy.prototxt": "https://raw.githubusercontent.com/chuanqi305/MobileNet-SSD/master/MobileNetSSD_deploy.prototxt",
    "MobileNetSSD_deploy.caffemodel": "https://raw.githubusercontent.com/chuanqi305/MobileNet-SSD/master/MobileNetSSD_deploy.caffemodel"
}

for name, url in urls.items():
    print(f"Downloading {name}...")
    urllib.request.urlretrieve(url, f"models/{name}")

print("Download complete. You can now run the detection script.")
```


Downloading MobileNetSSD_deploy.prototxt...
Downloading MobileNetSSD_deploy.caffemodel...
Download complete. You can now run the detection script.

```
: prototxt = "models/MobileNetSSD_deploy.prototxt"
  caffemodel = "models/MobileNetSSD_deploy.caffemodel"

  if not os.path.exists(prototxt) or not os.path.exists(caffemodel):
      print("Model files missing!")
  else:
      net = cv2.dnn.readNetFromCaffe(prototxt, caffemodel)
      print("Model Loaded Successfully")
```

Model Loaded Successfully

3. Class Labels

```
CLASSES = ["background", "aeroplane", "bicycle", "bird", "boat",
            "bottle", "bus", "car", "cat", "chair", "cow", "diningtable",
            "dog", "horse", "motorbike", "person", "pottedplant",
            "sheep", "sofa", "train", "tvmonitor"]
```

4. Static Image Detection

```
frame = cv2.imread("imagee.jfif")

if frame is None:
    print("Image not found!")
else:
    (h, w) = frame.shape[:2]

    # Preprocessing the image into a blob
    blob = cv2.dnn.blobFromImage(
        cv2.resize(frame, (300, 300)),
        0.007843,
        (300, 300),
        127.5
    )

    net.setInput(blob)
    detections = net.forward()
```

```

for i in range(detections.shape[2]):
    confidence = detections[0, 0, i, 2]

    if confidence > 0.5:
        idx = int(detections[0, 0, i, 1])
        box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
        (startX, startY, endX, endY) = box.astype("int")

        label = f"{CLASSES[idx]}: {confidence * 100:.1f}%"

        # Draw rectangle and text
        cv2.rectangle(frame, (startX, startY), (endX, endY), (0, 255, 0), 2)
        cv2.putText(frame, label, (startX, startY - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

# Show in notebook using Matplotlib
plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()

```



5. Webcam Real-Time Detection

```
# Use index 0 for default webcam; pdf shows index 8 was used in that environment
cap = cv2.VideoCapture(0)
```

```
while True:
    ret, frame = cap.read()
    if not ret:
        break

    (h, w) = frame.shape[:2]
    blob = cv2.dnn.blobFromImage(
        cv2.resize(frame, (300, 300)),
        0.007843,
        (300, 300),
        127.5
    )

    net.setInput(blob)
    detections = net.forward()

    for i in range(detections.shape[2]):
        confidence = detections[0, 0, i, 2]
```

```
        if confidence > 0.5:
            idx = int(detections[0, 0, i, 1])
            box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
            (startX, startY, endX, endY) = box.astype("int")

            label = f"{CLASSES[idx]}: {confidence * 100:.1f}%"

            cv2.rectangle(frame, (startX, startY), (endX, endY), (0, 255, 0), 2)
            cv2.putText(frame, label, (startX, startY - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

            cv2.imshow("Detection", frame)

            if cv2.waitKey(1) & 0xFF == ord('q'):
                break

cap.release()
cv2.destroyAllWindows()
```

Lab 6: Image Thresholding

Lab 6 introduces image thresholding, a basic technique used for image segmentation. It shows how a grayscale image can be converted into a binary (black-and-white) image to separate objects from the background based on pixel intensity. Students load an image using OpenCV and convert it to grayscale. Then, a binary threshold is applied where pixels above a certain value (127) become white, and those below it become black. This highlights the main object while removing unnecessary details. The lab displays the original image, grayscale version, and final binary result for comparison, helping students clearly understand the effect of thresholding. Overall, this lab demonstrates a simple yet important preprocessing step and encourages students to think about how threshold values and lighting conditions affect results.

TASK 1

Importing Libraries

```
: import cv2
import numpy as np
from matplotlib import pyplot as plt
from google.colab import files
```

Uploading Image

```
: uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving images.jpg to images.jpg

Reading and Preparing Image

```
: # Get the uploaded filename and read it
file_name = list(uploaded.keys())[0]
image = cv2.imread(file_name)

# Convert to RGB for visualization
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

Mean Shift Segmentation

```
# Set spatial radius (sp) and color radius (sr)
sp = 20
sr = 40

# Apply Mean Shift Filtering
shifted_image = cv2.pyrMeanShiftFiltering(image, sp, sr)
shifted_rgb = cv2.cvtColor(shifted_image, cv2.COLOR_BGR2RGB)
```

Displaying Mean Shift Results

```
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(shifted_rgb)
plt.title(f"Mean Shift (sp={sp}, sr={sr})")
plt.axis('off')
plt.show()
```

Original Image



Mean Shift (sp=20, sr=40)



TASK 2

Converting to Grayscale

```
# Convert the original image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

Applying Thresholding

```
# Set threshold value
threshold_val = 127

# Apply Binary Threshold to segment object from background
ret, thresh_img = cv2.threshold(gray, threshold_val, 255, cv2.THRESH_BINARY)
```

Displaying Thresholding Results

```
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Grayscale Image")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(thresh_img, cmap='gray')
plt.title(f"Thresholded Image (Val={threshold_val})")
plt.axis('off')
plt.show()
```

Grayscale Image



Thresholded Image (Val=127)



Lab 7: Unsupervised Image Segmentation

Lab 7 introduces an unsupervised image segmentation pipeline that divides an image into meaningful regions without using labeled data. It combines three main techniques: smoothing, superpixel segmentation, and region merging. First, the image is preprocessed using a bilateral filter to reduce noise while preserving edges. Then, the SLIC algorithm is applied to divide the image into small regions called superpixels. These superpixels group similar pixels together based on color and proximity. Next, a Region Adjacency Graph (RAG) is created, where neighboring regions are connected and compared based on color similarity. Similar regions are merged using a threshold, reducing many small segments into fewer meaningful regions. Finally, the results are displayed by comparing the original image, the segmented superpixels, and the final merged output. This lab helps students understand how images can be segmented using clustering and graph-based methods, forming a foundation for advanced image analysis tasks.

Task 1:

```
from skimage import io, segmentation, color
from skimage.graph import rag_mean_color, cut_threshold
from skimage.segmentation import slic, mark_boundaries
import matplotlib.pyplot as plt
from google.colab import files
```

```
# Step 1: Upload image
uploaded = files.upload()
img_path = list(uploaded.keys())[0]
img = io.imread(img_path)
```

Choose Files No file chosen

Upload widget is only available when the cell has been

executed in the current browser session. Please rerun this cell to enable.

Saving images (1).jpg to images (1).jpg

```
# Step 2: Create superpixels with SLIC
labels = slic(img, n_segments=200, compactness=10, start_label=0)
```

```
# Step 3: Build the RAG (using mean color)
rag = rag_mean_color(img, labels)
```

```
# Step 4: Merge regions using a color similarity threshold
labels_merged = cut_threshold(labels, rag, 35) # threshold can be tuned
```

```
# Step 5: Visualize
out = color.label2rgb(labels_merged, img, kind='overlay')

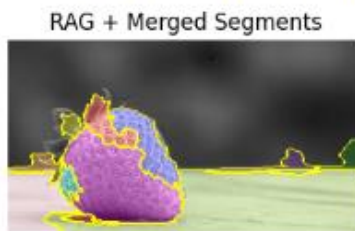
plt.figure(figsize=(12,6))

plt.subplot(1,3,1)
plt.imshow(mark_boundaries(img, labels))
plt.title("Original Superpixels")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(mark_boundaries(out, labels_merged))
plt.title("RAG + Merged Segments")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(img)
plt.title("real img")
plt.axis("off")
```

```
(np.float64(-0.5), np.float64(303.5), np.float64(165.5), np.float64(-0.5))
```

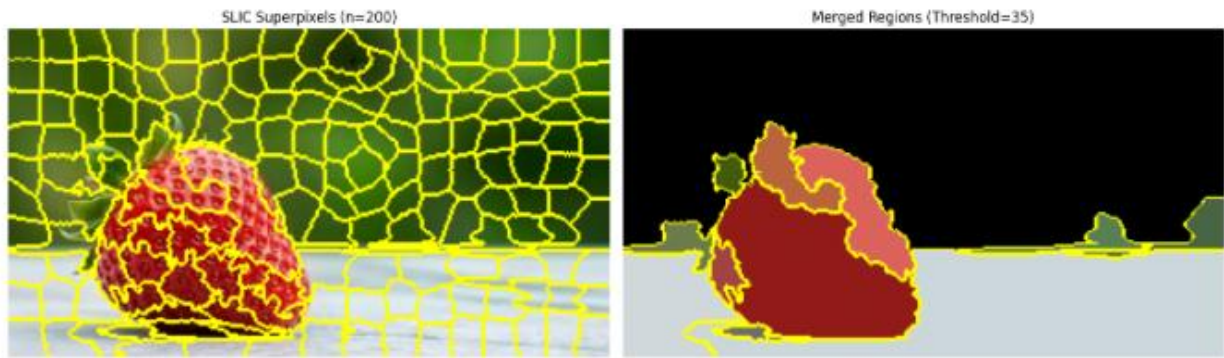


```
rag = rag_mean_color(img, labels)
thresh = 35
labels_merged = cut_threshold(labels, rag, thresh)
out = color.label2rgb(labels_merged, img, kind='avg')
plt.figure(figsize=(15, 8))

plt.subplot(1, 2, 1)
plt.imshow(mark_boundaries(img, labels))
plt.title(f"SLIC Superpixels (n=200)")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(mark_boundaries(out, labels_merged))
plt.title(f"Merged Regions (Threshold={thresh})")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Task 2:

```
import matplotlib.pyplot as plt
from skimage import io, segmentation, color, graph
from skimage.segmentation import slic, mark_boundaries
from skimage.graph import rag_mean_color, cut_threshold

# Step 1: Load the image
# Note: Ensure the image path matches the one used in your environment
img_path = list(uploaded.keys())[0]
img = io.imread(img_path)

# Step 2: Generate Superpixels using the SLIC method
# n_segments defines the approximate number of labels
# compactness balances color similarity and proximity
labels = slic(img, n_segments=200, compactness=10, start_label=1)

# Step 3: Construct a Region Adjacency Graph (RAG)
# This uses the mean color of each superpixel to build the graph
g = rag_mean_color(img, labels)

# Step 4: Merge regions based on a color similarity threshold
# Adjacent regions with color difference below 'thresh' will be combined
thresh = 35
labels_merged = cut_threshold(labels, g, thresh)

# Step 5: Process for Display
# Color the segments with their average color for the merged result
out_merged = color.label2rgb(labels_merged, img, kind='avg')
```

```

# Step 6: Visualization
plt.figure(figsize=(18, 6))

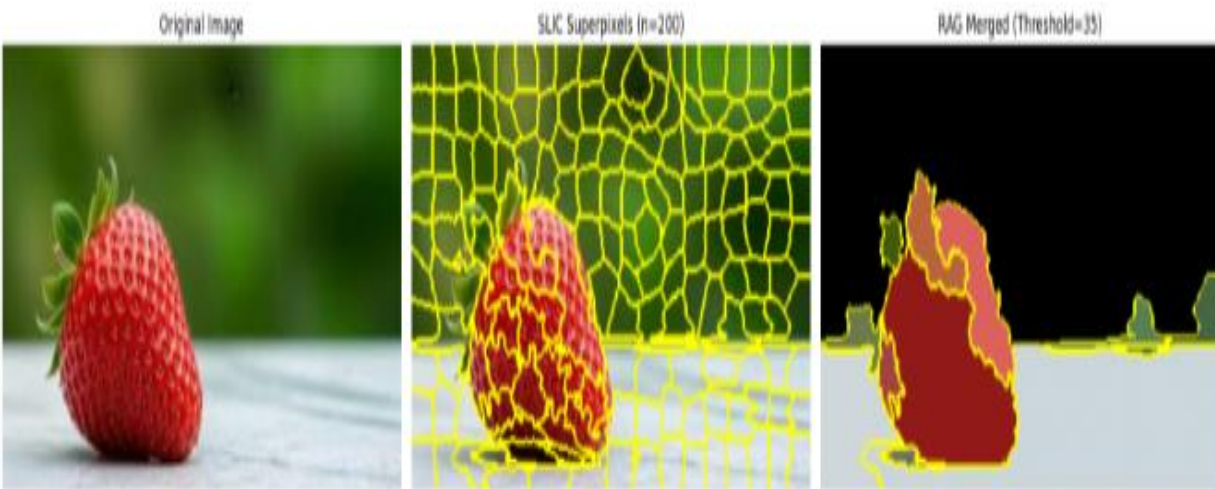
# Subplot 1: Original Image
plt.subplot(1, 3, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")

# Subplot 2: SLIC Superpixels
plt.subplot(1, 3, 2)
plt.imshow(mark_boundaries(img, labels))
plt.title("SLIC Superpixels (n=200)")
plt.axis("off")

# Subplot 3: RAG Merged Result
plt.subplot(1, 3, 3)
plt.imshow(mark_boundaries(out_merged, labels_merged))
plt.title(f"RAG Merged (Threshold={thresh})")
plt.axis("off")

plt.tight_layout()
plt.show()

```



Task 3:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import io, segmentation, color
from skimage.graph import rag_mean_color, cut_threshold
from skimage.segmentation import slic, watershed, mark_boundaries
from scipy import ndimage as ndi
from google.colab import files
```

```
# --- Step 0: Upload Image ---
# Colab-specific method to handle input files
uploaded = files.upload()
img_path = list(uploaded.keys())[0]
img = cv2.imread(img_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Choose Files

No file chosen

Upload widget is only available when the cell has been

executed in the current browser session. Please rerun this cell to enable.

Saving images (1).jpg to images (1).jpg

```
# --- Step 1: Mean Shift Filtering (Noise Reduction & Smoothing) ---
# Intent: Reduce noise and smooth color regions while preserving edges
# sp: spatial radius, sr: color radius
mean_shift = cv2.pyrMeanShiftFiltering(img, sp=20, sr=40)
mean_shift_rgb = cv2.cvtColor(mean_shift, cv2.COLOR_BGR2RGB)

# --- Step 2: SLIC Superpixels (Input: Mean Shift Result) ---
# Intent: Generate initial superpixel segments based on smoothed regions
labels_slic = slic(mean_shift_rgb, n_segments=250, compactness=10, start_label=1)

# --- Step 3: RAG Region Merging (Form Region-based Clusters) ---
# Intent: Group superpixels into clusters using color similarity
rag = rag_mean_color(mean_shift_rgb, labels_slic)
labels_merged = cut_threshold(labels_slic, rag, 35) # 35 is the color similarity threshold
```



```

# --- Step 4: Watershed (Boundary Refinement & Separation) ---
# Intent: Use clusters to refine boundaries and separate touching objects
gray = cv2.cvtColor(mean_shift, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Distance transform helps find centers of objects to separate them
distance = ndi.distance_transform_edt(thresh)
# Apply watershed using the RAG merged clusters as markers
labels_refined = watershed(-distance, labels_merged, mask=thresh)

# --- Step 5: Visualization ---
out_initial = mark_boundaries(mean_shift_rgb, labels_slic)
out_merged = color.label2rgb(labels_merged, img_rgb, kind='avg')
out_refined = color.label2rgb(labels_refined, img_rgb, kind='avg')

plt.figure(figsize=(20, 10))

plt.subplot(1, 3, 1)
plt.imshow(out_initial)
plt.title("1. Mean Shift + SLIC")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(mark_boundaries(out_merged, labels_merged))
plt.title("2. RAG Region Merging")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(mark_boundaries(out_refined, labels_refined))
plt.title("3. Watershed Boundary Refinement")
plt.axis("off")

plt.tight_layout()
plt.show()

```



Lab 8: Felzenszwalb Graph-Based Segmentation

Lab 8 focuses on unsupervised image segmentation using Felzenszwalb's graph-based algorithm. The main goal is to understand how the **scale parameter** affects the segmentation results. The process starts by loading and preparing the image, followed by applying a bilateral filter to reduce noise while preserving edges. The Felzenszwalb algorithm is then applied with different scale values (50, 100, 200, and 400). For each scale, the number and size of segments are calculated. Students observe that smaller scale values produce many small segments, while larger values result in fewer and larger regions. The results are displayed visually, allowing easy comparison of how segmentation changes with scale. Overall, this lab helps students understand how parameter tuning affects graph-based segmentation and prepares them for more advanced image analysis techniques.

Task 1:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ————— 4s 0us/step

x_train, x_test = x_train / 255.0, x_test / 255.0

model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Conv2D(128, (3, 3), activation='relu'),

    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```



```
history = model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))

Epoch 1/5
1563/1563 ————— 575s 367ms/step - accuracy: 0.5153 - loss: 1.3610 - val_accu
acy: 0.6092 - val_loss: 1.1115
Epoch 2/5
1563/1563 ————— 570s 365ms/step - accuracy: 0.6716 - loss: 0.9413 - val_accu
acy: 0.6691 - val_loss: 0.9593
Epoch 3/5
1563/1563 ————— 569s 364ms/step - accuracy: 0.7376 - loss: 0.7667 - val_accu
acy: 0.7007 - val_loss: 0.8739
Epoch 4/5
1563/1563 ————— 568s 363ms/step - accuracy: 0.7813 - loss: 0.6306 - val_accu
acy: 0.7088 - val_loss: 0.8709
Epoch 5/5
1563/1563 ————— 552s 353ms/step - accuracy: 0.8254 - loss: 0.5053 - val_accu
acy: 0.6972 - val_loss: 0.9735

test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f'\nFinal Test Accuracy: {test_acc:.4f}')

313/313 - 28s - 90ms/step - accuracy: 0.6972 - loss: 0.9735

Final Test Accuracy: 0.6972
```

Task 2:

```
import numpy as np
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions
from tensorflow.keras.preprocessing import image
```

```
model = MobileNetV2(weights='imagenet')
```

```
img_path = '/content/img.jpg'
img = image.load_img(img_path, target_size=(224, 224))
x = image.img_to_array(img)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)
```

```
preds = model.predict(x)
decoded_results = decode_predictions(preds, top=3)[0]
```

```
1/1 ————— 3s 3s/step
```

```
for i, (imagenet_id, label, prob) in enumerate(decoded_results):
    print(f'{i + 1}. {label}: {prob:.2%}')
```

```
1. tiger_cat: 80.20%
2. tabby: 11.03%
3. Egyptian_cat: 1.57%
```

Lab 9: Image Segmentation Pipeline

Lab 9 presents an advanced image segmentation pipeline that combines multiple techniques for better object separation. The process starts by smoothing the input image using bilateral filtering, which reduces noise while preserving edges. Next, Felzenszwalb's graph-based method is applied to divide the image into smaller regions (over-segmentation). These regions are then used as markers for the watershed algorithm. A gradient map is created using the Sobel operator, and watershed segmentation is applied to merge regions and refine object boundaries. The results are shown step by step, including the original image, smoothed image, segmented regions, and final output. Overall, this lab demonstrates how combining different segmentation techniques can improve accuracy and produce meaningful object boundaries, making it useful for real-world applications like

```
from skimage import io, segmentation, color, filters, morphology
from skimage.segmentation import mark_boundaries, watershed
from skimage.restoration import denoise_bilateral
from skimage.feature import canny
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import numpy as np
from scipy import ndimage as ndi
from google.colab import files

# — Step 1: Upload image —————

uploaded = files.upload()
img_path = list(uploaded.keys())[0]
img = io.imread(img_path)
if img.ndim == 2:
    img = np.stack([img] * 3, axis=-1)
if img.shape[2] == 4:
    img = img[:, :, :3]

print(f"Image loaded: {img.shape} dtype={img.dtype}")
img_float = img.astype(np.float64) / 255.0
img_smoothed = denoise_bilateral(
```

object detection and image analysis.

```

img_smoothed = denoise_bilateral(
    img_float,
    sigma_color=0.1,
    sigma_spatial=5,
    channel_axis=-1
)
img_ms = (img_smoothed * 255).clip(0, 255).astype(np.uint8)
print("Mean Shift smoothing complete.")
seg_felz = segmentation.felzenszwalb(img_ms, scale=150, sigma=0.5, min_size=60)
n_felz = len(np.unique(seg_felz))
print(f"Felzenszwalb: {n_felz} segments (used as Watershed markers)")
gray = color.rgb2gray(img_ms)
gradient = filters.sobel(gray)
markers = ndi.label(seg_felz)[0]
seg_ws = watershed(gradient, markers=markers, compactness=0.001)
n_ws = len(np.unique(seg_ws))
print(f"Watershed: {n_ws} refined regions")
felz_colored = color.label2rgb(seg_felz, img_ms, kind='avg')
ws_colored = color.label2rgb(seg_ws, img_ms, kind='avg')

fig, axes = plt.subplots(2, 3, figsize=(18, 11))
fig.suptitle(
    "Task 3 – Mean Shift → Felzenszwalb → Watershed\n"
    "Refining boundaries & separating touching objects",
    fontsize=13, fontweight='bold'
)

axes[0, 0].imshow(img)
axes[0, 0].set_title("Original Image")
axes[0, 0].axis("off")

axes[0, 1].imshow(img_ms)
axes[0, 1].set_title("After Mean Shift\n(noise reduced, colors smoothed)")
axes[0, 1].axis("off")

axes[0, 2].imshow(gradient, cmap='gray')
axes[0, 2].set_title("Sobel Gradient\n(Watershed elevation surface)")
axes[0, 2].axis("off")

axes[1, 0].imshow(mark_boundaries(felz_colored, seg_felz, color=(1, 0.5, 0)))
axes[1, 0].set_title(f"Felzenszwalb Segments\n({n_felz} regions → Watershed

```

```

markers)")
axes[1, 0].axis("off")

axes[1, 1].imshow(mark_boundaries(ws_colored, seg_ws, color=(0, 1, 1)))
axes[1, 1].set_title(f"Watershed Result\n({n_ws} refined regions)")
axes[1, 1].axis("off")

boundary_mask = segmentation.find_boundaries(seg_ws, mode='outer')
overlay = img.copy()
overlay[boundary_mask] = [255, 0, 0]
axes[1, 2].imshow(overlay)
axes[1, 2].set_title("Watershed Boundaries\nOverlaid on Original")
axes[1, 2].axis("off")

plt.tight_layout()
plt.show()

_, ws_counts = np.unique(seg_ws, return_counts=True)
_, felz_counts = np.unique(seg_felz, return_counts=True)
total_px = img.shape[0] * img.shape[1]

print(f"\n{'-'*50}")

```

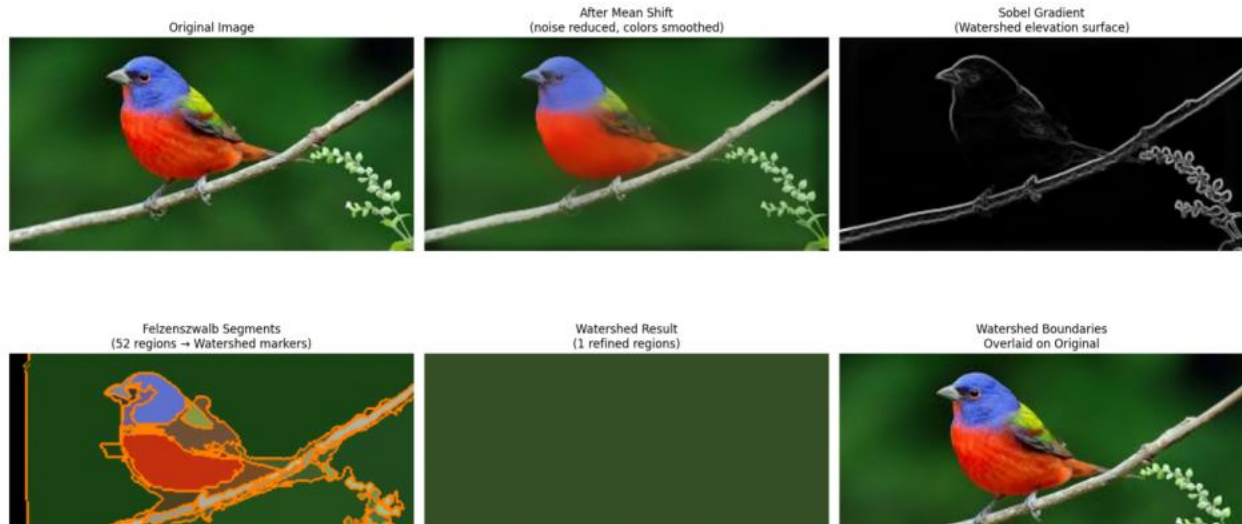


```

print('-'*50)
print(f"{'Felzenszwalb':<25} {n_felz:>8} {felz_counts.mean():>10.1f}")
print(f"{'Watershed':<25} {n_ws:>8} {ws_counts.mean():>10.1f}")
print('-'*50)
print(f"Total pixels: {total_px:,}")

```

Task 3 — Mean Shift → Felzenszwalb → Watershed
Refining boundaries & separating touching objects



Lab 10: CIFAR-10 Image Classification using CNN

Lab 10 focuses on image classification using a Convolutional Neural Network (CNN) on the CIFAR-10 dataset, which contains 60,000 small color images from 10 different classes. The process begins by loading and normalizing the data. A CNN model is then built using multiple convolutional layers with ReLU activation, followed by max-pooling, flattening, and dense layers for classification. The model is trained for a few epochs using the Adam optimizer and cross-entropy loss. During training, accuracy improves significantly, but validation accuracy is lower, indicating slight overfitting. Overall, this lab demonstrates how CNNs can be used for multi-class image classification and highlights the importance of improving models using techniques like dropout or data augmentation.

Load library

```
import tensorflow as tf
from tensorflow.keras import layers, models
# Load dataset (CIFAR-10)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
# Normalize
x_train, x_test = x_train/255.0, x_test/255.0
# Build CNN
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
# Compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
# Train
model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
# Evaluate
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)
```

```
Epoch 1/5
1563/1563 ————— 471s 300ms/step - accuracy: 0.4825 - loss:
Epoch 2/5
1563/1563 ————— 434s 278ms/step - accuracy: 0.6503 - loss:
Epoch 3/5
1563/1563 ————— 433s 272ms/step - accuracy: 0.7202 - loss:
Epoch 4/5
1563/1563 ————— 417s 267ms/step - accuracy: 0.7679 - loss:
Epoch 5/5
1563/1563 ————— 442s 283ms/step - accuracy: 0.8118 - loss:
313/313 ————— 23s 74ms/step - accuracy: 0.7009 - loss: 0.9
Test Accuracy: 0.7009000182151794
```

Lab 11: Image Classification using Pre-trained CNN (MobileNetV2)

Lab 11 demonstrates transfer learning for image classification using a pre-trained CNN called MobileNetV2, which is trained on the ImageNet dataset containing over 1.4 million images. The lab shows how to load images from local files or URLs, preprocess them by resizing to 224×224 and converting them into the required format, and then feed them into the pre-trained model for prediction. The model outputs probabilities for different classes, and the top predictions are displayed along with the input image. Tests are performed on various images such as animals, vehicles, and food items. Overall, this lab highlights the power of transfer learning, showing how a pre-trained model can accurately classify new images without additional training, making it highly practical for real-world applications.

Load library:

```
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input,
decode_predictions
import numpy as np
import matplotlib.pyplot as plt
import requests
from PIL import Image
import io
from urllib.parse import urlparse
import urllib.request
# Load pre-trained MobileNetV2 model with ImageNet weights
print("Loading MobileNetV2 with ImageNet weights...")
model = MobileNetV2(weights='imagenet', input_shape=(224, 224, 3))
print("Model loaded successfully!")
def load_and_preprocess_image(img_path, target_size=(224, 224)):
    """
```

Load image:

```
Improved function to load image from various sources with better error
handling
"""
try:
    # Check if it's a URL
    if img_path.startswith(('http://', 'https://')):
        # Create a request with a User-Agent header to avoid blocking
        headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36'}
        req = urllib.request.Request(img_path, headers=headers)
```

```

    try:
        with urllib.request.urlopen(req, timeout=10) as response:
            img_data = response.read()
            img = Image.open(io.BytesIO(img_data))
    except Exception as e:
        # Fallback using requests
        print(f"URL download failed, trying alternative method: {e}")
        response = requests.get(img_path, headers=headers, timeout=10)
        img = Image.open(io.BytesIO(response.content))

# Check if it's a local file path

```

Convert to RGB:

```

else:
    img = Image.open(img_path)

    # Convert to RGB if necessary
    if img.mode != 'RGB':
        img = img.convert('RGB')

    # Resize image
    img = img.resize(target_size, Image.Resampling.LANCZOS)

    # Convert to array
    x = np.array(img)

```

CONVERT TO
ARRAY:

```

# Convert to array
x = np.array(img)

# Expand dimensions to match model input
x = np.expand_dims(x, axis=0)

# Preprocess input (specific to MobileNetV2)
x = preprocess_input(x)

return x, img

except Exception as e:
    print(f"Error loading image: {e}")

```


LOAD AND PREPROCESS IMAGE:

```
        raise
def classify_image(img_path, model, top_k=3):
    """
    Classify an image using the pre-trained model and display top k predictions
    """
    try:
        # Load and preprocess image
        x, original_img = load_and_preprocess_image(img_path)

        # Make prediction
        predictions = model.predict(x, verbose=0)

        # Decode predictions
        decoded_predictions = decode_predictions(predictions, top=top_k)[0]

        # Create visualization
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

        # Show original image
        ax1.imshow(original_img)
        ax1.set_title("Uploaded Image", fontsize=14, fontweight='bold')
        ax1.axis('off')
```

PREDICTION AS BAR CHART:

```
# Show predictions as bar chart
labels = [pred[1] for pred in decoded_predictions]
probabilities = [pred[2] for pred in decoded_predictions]

# Create color gradient
colors = plt.cm.viridis(np.linspace(0.2, 0.8, top_k))

y_pos = np.arange(len(labels))
bars = ax2.barh(y_pos, probabilities, color=colors)
ax2.set_yticks(y_pos)
ax2.set_yticklabels(labels, fontsize=10)
ax2.set_xlabel('Probability', fontsize=12, fontweight='bold')
ax2.set_title(f'Top {top_k} Predictions', fontsize=14, fontweight='bold')
ax2.invert_yaxis() # Highest probability at top
```



```

# Add probability values on bars
for i, (bar, prob) in enumerate(zip(bars, probabilities)):
    width = bar.get_width()
    ax2.text(width + 0.01, bar.get_y() + bar.get_height()/2,
             f'{prob:.3f} ({prob*100:.1f}%)',
             ha='left', va='center', fontsize=10, fontweight='bold')

plt.tight_layout()
plt.show()

# Print predictions in a formatted way
print(f"\n{'='*60}")
print("CLASSIFICATION RESULTS")
print(f"{'='*60}")
for i, (_, label, prob) in enumerate(decoded_predictions, 1):
    print(f"{i}. {label:20s} : {prob:.3f} ({prob*100:.1f}%)")
    print(f"    {'-' * 40}")

return decoded_predictions

except Exception as e:
    print(f"Classification failed: {e}")
    return None

# Test with reliable image URLs (working examples)
print("\n" + "="*60)
print("EXAMPLE 1: Classifying a Dog Image")
print("="*60)

# Using a more reliable image URL (GitHub hosted or different source)
dog_url = "https://images.dog.ceo/breeds/shiba/shiba-14.jpg" # Reliable dog API
classify_image(dog_url, model, top_k=3)

print("\n" + "="*60)
print("EXAMPLE 2: Classifying a Cat Image")
print("="*60)
cat_url = "https://cdn2.thecatapi.com/images/9j5.jpg" # Reliable cat API
classify_image(cat_url, model, top_k=3)

print("\n" + "="*60)
print("EXAMPLE 3: Classifying a Vehicle")
print("="*60)
car_url = "https://www.caranddriver.com/uploads/2022/04/2023-Honda-CR-V-front-
three-quarter.jpg"
classify_image(car_url, model, top_k=3)

```

```

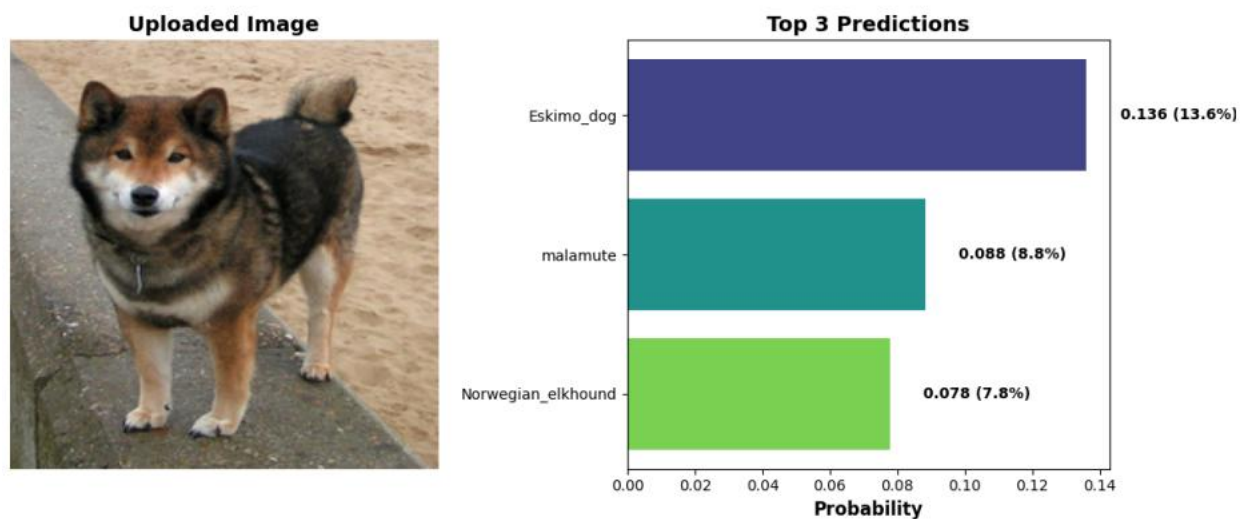
classify_image(cat_url, model, top_k=3)

print("\n" + "="*60)
print("EXAMPLE 2: Classifying a Cat Image")
print("="*60)
cat_url = "https://cdn2.thecatapi.com/images/9j5.jpg" # Reliable cat API
classify_image(cat_url, model, top_k=3)

print("\n" + "="*60)
print("EXAMPLE 3: Classifying a Vehicle")
print("="*60)
car_url = "https://www.caranddriver.com/uploads/2022/04/2023-Honda-CR-V-front-
three-quarter.jpg"
classify_image(car_url, model, top_k=3)

print("\n" + "="*60)
print("EXAMPLE 4: Classifying Food")
print("="*60)
pizza_url =
"https://www.foodandwine.com/thmb/4qg5tjuzYjSHmtZ8C5bVwT5uYak=/1500x0/filters:no_
upscale():max_bytes(150000):strip_icc()/classic-cheese-pizza-FT-RECIPE0422-
31a2c938fc2546c9a07b7011658cfd05.jpg"
classify_image(pizza_url, model, top_k=3)

```



```

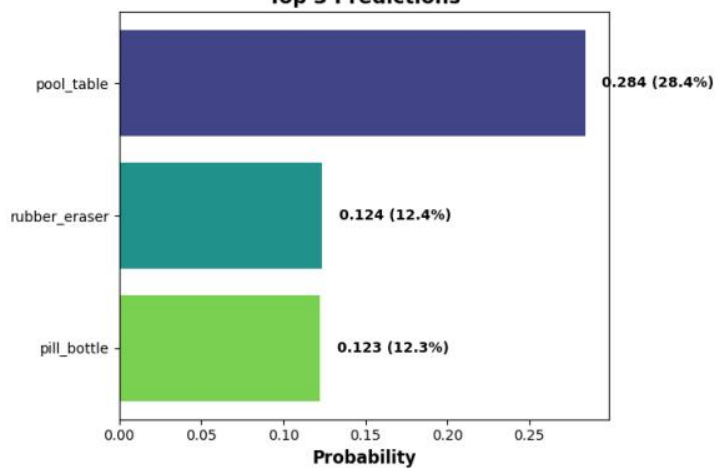
=====
CLASSIFICATION RESULTS
=====
1. Eskimo_dog           : 0.136 (13.6%)
-----
2. malamute             : 0.088 (8.8%)

```

Uploaded Image



Top 3 Predictions



=====
CLASSIFICATION RESULTS
=====

1. pool_table : 0.284 (28.4%)

2. rubber_eraser : 0.124 (12.4%)

3. pill_bottle : 0.123 (12.3%)

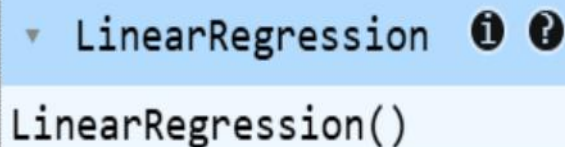
Lab 12: Simple Linear Regression with Scikit-Learn

This lab provides a concise and practical introduction to linear regression using the Scikit-Learn library. The workflow begins with importing the necessary modules: NumPy for numerical operations, Matplotlib for visualization, and the LinearRegression class from Scikit-Learn. A simple synthetic dataset is created, where study hours serve as the input feature (X) and corresponding exam marks as the target variable (y). The features are properly reshaped to meet the library's requirements, and the model is trained using the `.fit()` method. After training, the model is used to predict exam marks for a new input value (7 hours of study), and the result is printed. The lab then visualizes both the actual data points as a scatter plot and the fitted regression line on the same graph, providing a clear visual confirmation of the linear relationship. This notebook is ideal for beginners to understand the complete supervised learning workflow: preparing data, training a model, making predictions, and evaluating results visually. It highlights the simplicity and efficiency of Scikit-Learn for basic machine learning tasks and sets a strong foundation for exploring more complex regression problems and evaluation metrics.

LOAD

LIBRARY:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
# Study hours and Marks
study_hours = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
marks = np.array([35, 40, 50, 55, 65, 70])
model = LinearRegression()
model.fit(study_hours, marks)
```



▼ LinearRegression ⓘ ?

LinearRegression()


```

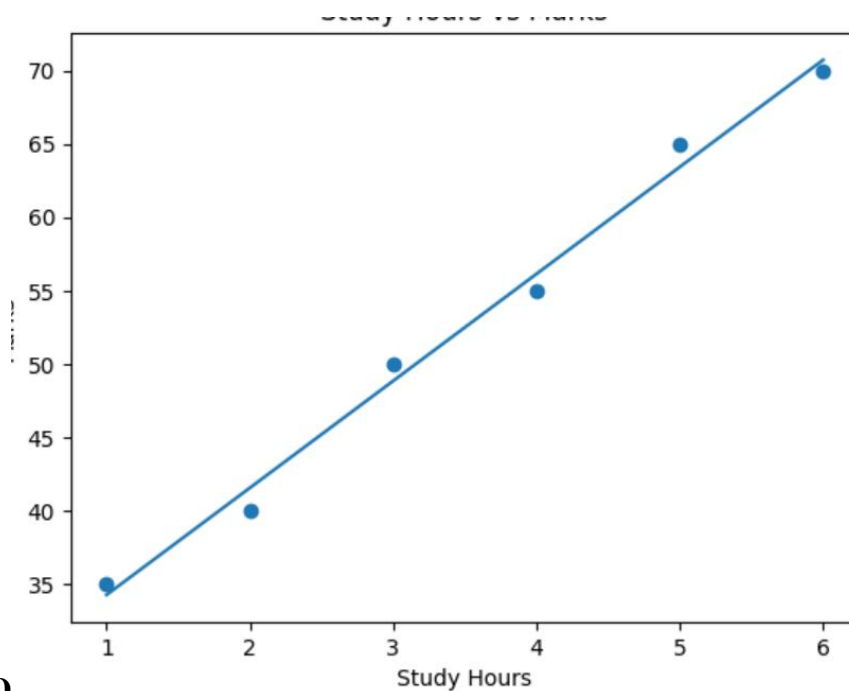
predicted_marks = model.predict(study_hours)

# Predict for new value
new_hours = np.array([[7]])

new_prediction = model.predict(new_hours)

print("Prediction for 7 hours:", new_prediction[0])
plt.scatter(study_hours, marks)      # actual data
plt.plot(study_hours, predicted_marks) # regression line
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Study Hours vs Marks")
plt.show()

```



FLO

WCHART:

Lab 13: Feature Extraction Using SIFT in OpenCV

This notebook introduces the Scale-Invariant Feature Transform (SIFT) algorithm for keypoint detection and feature extraction using OpenCV. The lab begins by allowing users to upload images via Google Colab's built-in file upload widget. Each uploaded image is read using OpenCV, converted to grayscale, and passed to `cv2.SIFT_create()` to initialize the SIFT detector. Keypoints and their corresponding local feature descriptors are extracted for each image, and the shapes of the descriptor matrices are printed to demonstrate feature dimensionality. The keypoints are then drawn on the original images and displayed side-by-side using Matplotlib. SIFT is a powerful computer vision technique that identifies distinctive image features that remain invariant to scaling, rotation, translation, and partial illumination changes. This makes SIFT particularly valuable for applications such as image stitching, object recognition, 3D reconstruction, and structure from motion. The lab provides hands-on experience with one of the most robust and widely used feature extraction methods, illustrating how local image features can be detected and represented for downstream computer vision tasks.

LOAD

LIBRARY:

```
import cv2
import matplotlib.pyplot as plt
from google.colab import files
# Upload images
uploaded = files.upload()
filenames = list(uploaded.keys())
# Load images
img1 = cv2.imread(filenames[0])
img2 = cv2.imread(filenames[1])

# Convert to grayscale
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
# SIFT
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(gray1, None)
```

CONVERT BGR TO RGB

```
kp2, des2 = sift.detectAndCompute(gray2, None)
print("Image 1 descriptors shape:", des1.shape)
print("Image 2 descriptors shape:", des2.shape)
# Draw keypoints
img1_sift = cv2.drawKeypoints(img1, kp1, None)
img2_sift = cv2.drawKeypoints(img2, kp2, None)
# Convert BGR → RGB
img1_sift = cv2.cvtColor(img1_sift, cv2.COLOR_BGR2RGB)
img2_sift = cv2.cvtColor(img2_sift, cv2.COLOR_BGR2RGB)
```

```
# Display BOTH
plt.figure(figsize=(10,6))

plt.subplot(1,2,1)
plt.imshow(img1_sift)
plt.title("Image 1 SIFT Keypoints")
plt.axis("off")

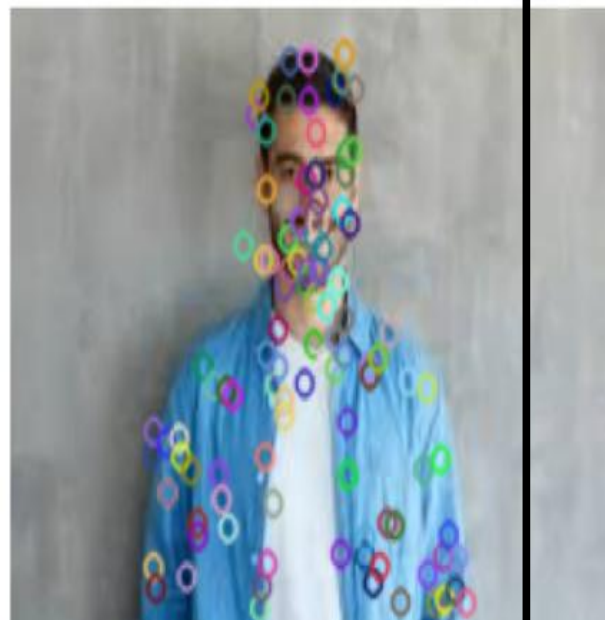
plt.subplot(1,2,2)
plt.imshow(img2_sift)
plt.title("Image 2 SIFT Keypoints")
plt.axis("off")

plt.show()
```

Image 1 SIFT Keypoints



Image 2 SIFT Keypoints



Lab 14: HOG Feature Extraction for Image Analysis

This lab demonstrates the use of Histogram of Oriented Gradients (HOG) for dense feature extraction in image analysis. After uploading an image using Google Colab's file interface, the image is read with OpenCV and converted to grayscale to simplify the gradient computation. The `hog()` function from `skimage.feature` is then applied, which divides the image into small spatial cells, computes gradient orientation histograms for each cell, and normalizes them across larger blocks to achieve illumination invariance. Key parameters such as `orientations`, `pixels_per_cell`, and `cells_per_block` are specified to control the granularity and dimensionality of the resulting feature vector. The total number of HOG features is printed, providing insight into the descriptor's size. The lab also visualizes the original grayscale image alongside a directional gradient image that represents the HOG features. HOG descriptors are widely used in object detection (e.g., pedestrian detection) and image classification tasks because they effectively capture edge and gradient structure while remaining invariant to small changes in illumination and contrast. This lab provides a practical introduction to a fundamental technique in handcrafted feature extraction.

LOAD LIBRARY

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure

# Upload image in Colab
from google.colab import files
uploaded = files.upload()

# Read image
image_path = list(uploaded.keys())[0]
image = cv2.imread(image_path)

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.imshow(gray, cmap='gray')
plt.title("Grayscale Image")
```

```
:  
plt.axis('off')
```

Grayscale Image



APPLY HOD:

```
# Apply HOG  
features, hog_image = hog(  
    gray,  
    orientations=9,      # Number of direction bins  
    pixels_per_cell=(8, 8),  
    cells_per_block=(2, 2),  
    visualize=True,  
    block_norm='L2-Hys'  
)  
  
print("HOG feature vector length:", len(features))  
# Rescale HOG image for better visualization  
hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 10))
```

```
# Plot original and HOG image
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(gray, cmap='gray')
plt.title("Original Image")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(hog_image_rescaled, cmap='gray')
plt.title("HOG Directional Features")
plt.axis('off')

plt.show()
```

Original Image



HOG Directional Features



Conclusion

This lab manual provided a comprehensive, hands-on journey through the fundamental concepts, classical techniques, and advanced deep learning architectures that form the backbone of modern computer vision. Spanning fourteen exercises, the labs progressively built proficiency across the entire computer vision pipeline—from basic image manipulation and predictive modeling to sophisticated feature extraction, object detection, and neural network-based classification.

The initial labs established essential skills in image processing using OpenCV, including reading, writing, and transforming images, applying geometric transformations, and performing essential filtering and edge detection operations. These foundational tasks are critical for any subsequent vision application, as they enable effective preprocessing and noise reduction.

Subsequent exercises introduced core feature extraction techniques. Labs on SIFT (Scale-Invariant Feature Transform) and HOG (Histogram of Oriented Gradients) demonstrated how to detect and describe local image features that remain invariant to scale, rotation, and illumination changes—key requirements for robust object recognition and image matching tasks. Feature matching and homography estimation were also explored, providing practical experience with image stitching and panorama creation.

